

Malware Analysis and Security

Monitoring Lab Deployment Report

1. Executive Summary

Lab Objectives

This lab provides a secure, heavily monitored environment designed for practical malware analysis and security monitoring. By deploying this infrastructure, security professionals can safely build foundational skills in identifying, analyzing, and detecting malicious software, such as RedLine Stealer.

Architecture Overview

The lab is divided into two primary environments to safely separate the analysis tools from the execution zone:

- **The Linux Environment (Blue Team):** Acts as the centralized hub. It hosts the Security Information and Event Management (SIEM) infrastructure for log storage and provides advanced command-line tools for static, dynamic, and network analysis.
- **The Windows Environment (Victim Machine):** The dedicated detonation zone where malware is safely executed and observed. It contains dynamic monitoring tools to track system behavior and generate telemetry.

Virtualization & Network Infrastructure

- **Hypervisor:** Oracle VirtualBox is used to host both virtual machines (VMs).
- **Network Isolation:** Both VMs operate within a single, **isolated internal network**. This air-gapped approach ensures that malware can be safely executed without risking the physical host machine or spreading to the external internet.

2. Linux Environment (Infrastructure & Analysis)

This environment serves as the backbone of the lab, hosting the centralized logging system and advanced analysis utilities.

2.1 Docker

Purpose: Runs applications inside isolated containers.

Use Case: Docker allows us to deploy SIEM components (Wazuh Manager, Dashboard, and Indexer) in isolated containers rather than directly on the Linux host. This prevents dependency conflicts and simplifies both installation and system cleanup.

Installation Steps:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Verification & Useful Commands:

```
docker --version
docker run hello-world

# Useful Management Commands
docker ps          # List running containers
docker ps -a       # List all containers
docker images      # List downloaded images
docker stop <container> # Stop a specific container
docker rm <container>  # Remove a specific container
```

2.2 Elasticsearch

Purpose: A distributed search and analytics engine used to store, index, and search security logs.

Use Case: Serving as the backend data engine for Kibana, Elasticsearch stores Sysmon telemetry and Windows Event Logs. It indexes malware-related events, enabling fast and centralized threat hunting.

Installation Steps:

```
sudo apt update
sudo apt install apt-transport-https openjdk-17-jdk wget curl gnupg -y

wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elasticsearch-8.x.list

sudo apt update
sudo apt install elasticsearch -y
```

Start Service & Verification:

```
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch
sudo systemctl status elasticsearch
curl -k https://localhost:9200
```

Important Directories:

- **Configuration:** `/etc/elasticsearch/`
- **Logs:** `/var/log/elasticsearch/`
- **Data:** `/var/lib/elasticsearch/`

2.3 Elastic Stack Components

Purpose: A comprehensive security monitoring platform used for log collection, visualization, and threat detection.

- **Elasticsearch:** Stores and indexes security logs.
- **Kibana:** Provides the graphical interface for dashboards, SIEM capabilities, and log visualization.
- **Elastic Agent:** Installed on the Windows endpoint to securely collect and forward logs.
- **Fleet:** A centralized management server for administering Elastic Agents.

Log Workflow:

Windows Execution → Sysmon → Elastic Agent → Elasticsearch → Kibana → Detection Rules → Alerts & Investigation

2.4 CAPA

Purpose: Identifies malware capabilities using predefined behavioral rules.

Use Case: Instead of manually analyzing complex API calls, CAPA automates the identification of what the malware is capable of doing. This includes actions like collecting system information, reading registries, stealing credentials, or establishing network communication.

Installation Steps:

```
# After downloading the latest Linux release from GitHub and extracting:  
chmod +x capa  
sudo mv capa /usr/local/bin/
```

Verification & Usage:

```
capa -version  
capa sample.exe  
capa sample.exe > capa_report.txt # Save output for reporting
```

2.5 FLOSS

Purpose: Extracts decoded and obfuscated strings from malware samples.

Use Case: Advanced malware hides its core functions. FLOSS uncovers obfuscated strings to reveal malicious URLs, domains, registry keys, and file paths. These details are critical for extracting Indicators of Compromise (IOCs) and writing YARA rules.

Installation Steps:

```
# After downloading the latest Linux release from GitHub and extracting:  
chmod +x floss  
sudo mv floss /usr/local/bin/
```

Verification & Usage:

```
floss -version  
floss sample.exe  
floss sample.exe > strings.txt # Save output for IOC extraction
```

2.6 Wireshark

Purpose: Captures and analyzes network traffic.

Use Case: Malware often attempts to communicate with a remote Command & Control (C2) server. Wireshark intercepts this traffic, allowing analysts to identify malicious DNS queries, external IP addresses, and HTTP/HTTPS protocols.

Installation Steps:

```
sudo apt update  
sudo apt install wireshark -y  
sudo usermod -aG wireshark $USER  
newgrp wireshark
```

Basic Analysis Filters:

- dns
- tcp
- http
- tls
- ip.addr == x.x.x.x

2.7 YARA

Purpose: Detects malware using custom rules based on strings, patterns, or binary characteristics.

Use Case: After identifying how a specific malware operates, analysts use YARA to create custom signatures. These rules can automatically detect future samples sharing the same indicators, drastically reducing manual analysis time.

Installation & Usage:

```
sudo apt update
sudo apt install yara -y
yara -version
yara rule.yar sample.exe
```

3. Windows Environment (Execution & Telemetry)

This environment serves as the primary detonation zone. It includes tools to inspect the malware statically before execution, as well as dynamically monitor its impact on the operating system.

3.1 Detect It Easy (DIE)

Purpose: A static analysis tool used to inspect executable files without running them.

Use Case: Before detonation, DIE quickly profiles the malicious file. It identifies critical metadata, including the file architecture, the compiler used, and whether the malware has been packed or obfuscated.

Setup & Usage:

1. Download the portable x64 version from GitHub releases.
2. Extract and place it in: `C:\Tools\DIE`
3. **Verification:** Run die.exe.
4. **Workflow:** Malware.exe → Detect It Easy → Static Information Gathering

3.2 Sysinternals Suite

Purpose: The official Microsoft toolkit for advanced system troubleshooting and dynamic malware analysis.

Main Tools Used:

- **Procmon (Process Monitor):** Tracks real-time file system, registry, and process activity.
- **Autoruns:** Identifies persistence mechanisms by revealing programs configured to start automatically upon boot.
- **Process Explorer:** An advanced task manager used to inspect active process trees and memory usage.
- **Handle & PsExec:** Command-line utilities for deep process and system manipulation.

3.3 Regshot

Purpose: A system registry comparison tool.

Use Case: Many malware families establish persistence by secretly modifying the Windows Registry. Regshot takes system snapshots before and after execution to highlight exactly

which registry keys were added, modified, or deleted.

Setup & Usage:

1. Download the x64 portable version from GitHub releases.
2. Extract and place it in: C:\Tools\Regshot
3. **Verification:** Run regshot-x64.exe.
4. **Analysis Workflow:** Take 1st Shot → Run Malware → Take 2nd Shot → Compare → Analyze Registry Changes

3.4 TCPView

Purpose: A real-time network monitoring application (included in the Sysinternals Suite).

Use Case: TCPView provides immediate visibility into network activity. It allows analysts to quickly spot unauthorized inbound and outbound connections created by a malicious process to identify potential C2 servers.

Usage:

1. Run Tcpview64.exe.
2. **Information Collected:** Process Name, Local Address, Remote Address, Remote Port, Connection State.
3. **Analysis Workflow:**
Run TCPView → Execute Malware → Identify C2 Connections → Analyze Traffic deeper with Wireshark (on Linux)

4. Summary and Conclusion

Summary

The deployment of this Malware Analysis and Security Monitoring Lab establishes a robust, air-gapped environment specifically tailored for investigating malicious activities. By dividing the architecture into a centralized Linux analysis hub and a Windows execution zone, the lab ensures threats can be observed safely. Furthermore, system telemetry and network traffic are seamlessly captured, forwarded, and stored for deep analysis.

Conclusion

Ultimately, this lab equips security practitioners with the hands-on capabilities required to dissect complex threats. The environment facilitates a complete, end-to-end understanding of the incident response lifecycle. Analysts begin with static inspection using DIE, FLOSS, and CAPA. They then transition to dynamic behavior monitoring with Sysinternals and Regshot, alongside network analysis via Wireshark and TCPView.

By actively routing this endpoint telemetry into the Elastic Stack, analysts can seamlessly

translate their findings and extracted Indicators of Compromise (IOCs) into actionable detection engineering. This allows for the creation of custom YARA rules and Kibana alerts, effectively bridging the gap between theoretical malware analysis and practical, real-world cyber defense.